
FVSpec: Real-World PBTs as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present a benchmark for evaluating AI models and agents on real-world formal
2 software verification tasks. We first scrape $\approx 20k$ property-based tests (PBTs) from
3 permissively licensed real-world Python repositories, then automatically translate
4 them into Lean 4 specifications with sorry placeholders. Translating PBTs into
5 Lean specifications is challenging: it requires modeling Python semantics in Lean,
6 inferring the logical property encoded in an imperative PBT, and handling the inher-
7 ent difficulties of dependently-typed programming in a seldom-used language. We
8 describe a three-agent LLM pipeline for transpiling PBTs into Lean specifications,
9 evaluate coverage and quality metrics, and provide baselines for proof generation
10 using several automated and model-based approaches. Our benchmark aims to
11 drive progress on the underexplored problem of AI-assisted formal verification of
12 real-world software, which we claim is of increasing importance as AI produces
13 more and more of the world’s code.

14 1 Introduction

15 Several AI safety proposals are premised on implementing safeguards by way of *formal verification*
16 (FV)¹ – that is, software systems that can produce and unequivocally certify mathematical proofs.
17 These systems can be split into *interactive theorem provers* (ITPs) which accept and validate user-
18 supplied proofs, and *automated theorem provers* (ATPs) which attempt (but often fail) to prove
19 theorems fully autonomously. ARIA’s Safeguarded AI [8], Guaranteed Safe AI [7], and Scalable
20 Formal Oversight [44, 33], among others, all suggest that in the future, all AI-generated software will
21 be formally verified by autonomous FV agents, who will steer ITPs in much the same way that
22 coding agents currently steer software production.

23 The only way FV can play this role is if agents can conduct verification work at least as well and as
24 quickly as they can produce code. Otherwise, FV becomes a bottleneck and will naturally wither under
25 the capital pressure to iterate². There is currently little evidence that such capabilities are possible.
26 To achieve them, we would first need high-quality benchmarks to train on. Existing benchmarks
27 (see Table 1) are too small [28], focused on advanced mathematics rather than engineering [15], or
28 derived from expert-driven verification projects [6] which are quite different from what you would
29 get were you to verify the average software project. To clarify this last point: historically, FV has
30 been bottle-necked by the number of human expert practitioners able to write proofs in an ITP³,
31 and therefore, FV was only ever applied to the most mathematically complex and safety-critical

¹FV forms a subset of *formal methods*, which also include PBT, model checking, and abstract interpretation.

²The reader is referred to [36] for a detailed discussion of how socioeconomic pressures degrade safety cultures and cause predictable accidents.

³We were unable to find any formal estimate of the number of human formal methods practitioners today. However, as a very rough proxy, de Almeida Borges et al. [9] conducted the largest ever survey of users of any particular ITP – in this case Coq, which is now called Rocq – at 466 participants. As a point of contrast, that year’s survey by the Rust foundation of Rust software engineers had 7,156 participants [39]. Anecdotally, most ITP work today is in Rocq, Lean 4, or Isabelle.

applications, such as compilers [25] or an OS kernel [23]. These projects are generally monolithic, closed systems, in contrast to the average software project which may have a frontend and backend, a database, reliance on several external black-box APIs such as an inference provider or a logging service, and an unnecessarily large dependency tree. The latter category of (normal) software is actually in many ways much more challenging to verify because it necessitates reasoning about black-box external systems and the interactions of several different components, potentially written in different languages and operating on different chips [4, 34].

MvH: A reviewer is going to ask us how we handle all this complexity. Do we have theorems that relate to external APIs? In what ways do we REALLY differ from other benchmarks? We need to substantiate this.

Benchmark	Language	Theorems	Focus
Minif2f [48]	Lean, Isabelle, HOL Light, Metamath	488	math Olympiad
PutnamBench [42]	Lean, Isabelle, Coq/Rocq	640	math Olympiad
Fimo [27]	Lean	149	math Olympiad
DafnyBench [28]	Dafny	782	scraped Dafny proofs
ProofNet [1]	Lean	371	U math
RLM25 [35]	Lean	619	G math
Lean Workbook [46]	Lean	57,000	HS, U, G math
FormalMATH [47]	Lean	5,560	math Olympiad
LEAN-GitHub [45]	Lean	28,597	scraped Lean proofs
ProverBench [37]	Lean	325	math Olympiad
MiniCTX [18]	Lean	762	six Lean projects
LeanGeo [38]	Lean	122	HS, U, G math
IneqMath [30]	Lean	1,522	math Olympiad
FATE- $\{M, H, X\}$ [21]	Lean	350	U, G math
VeriBench [3]	Lean	857	software verification
FVAPPS [13]	Lean	4,715	software verification (from APPS)
CLEVER [40]	Lean	161	software synthesis & verification
Vericoding [5]	Lean, Dafny, Verus	12,504	software synthesis & verification
MUSTARD [19]	Lean	5,866	HS, U math
FormL4 [29]	Lean	14,510	U math
CoqStoq (Rango) [41]	Coq/Rocq	10,396	software verification
MSC-180 [26]	Lean	180	U, G math
HOList [2]	HOL Light	29,462	U, G math

Table 1: **Prior ITP benchmarks.** HS = high-school, U = undergraduate, G = graduate. If one benchmark improves upon another, we exclude the lesser.

In light of the limitations of prior works, we generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a “close sibling” technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(1st)` always returns a sorted permutation of its input `1st`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams [12]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering, and provides only statistical assurance.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s [11], any unproven theorem statement is automatically treated as a PBT until the author supplies a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard. We build on this. First, we scrape $\approx 20k$ permissively-licensed PBTs written in Python’s Hypothesis framework [31]. Each PBT in our dataset can be seen as a formal conjecture about a piece of Python code, which the author would like to either substantiate (prove) or disprove. To make it possible for an AI to resolve these conjectures, we translate both code and PBTs to the theorem prover Lean, which

is becoming the de-facto standard in AI formal verification. Once the Lean version of the conjecture is (eventually) proven, it can be called a *theorem*. Until then, it has a `sorry` placeholder where the proof belongs. This approach was pioneered by [13] on the FVAPPS verification benchmark. In FVAPPS, source programs were taken from the APPS benchmark by [17], translated to PBTs, and then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

We use AI agents to translate from Python to Lean, in the style of FVAPPS. This process is necessarily lossy, that is to say, it does not guarantee strict equivalence. However, strict equivalence is only needed if the goal is to aid the authors of the original software by resolving their open conjectures. Although when applicable we consider this goal a positive secondary contribution of our work, our actual motivation is to build a large benchmark of realistic and high-quality software verification exercises, representative of the diversity of code found in “normal” codebases. Thus, we don’t worry whether or not our translations are perfectly equivalent. The result is tens of thousands of verification challenges. Each consists of four artifacts (Figure 1): (1) the original Python implementation, (2) the original Python PBT exercising it, (3) a complete Lean implementation of the same function (no `sorry`), and (4) a Lean theorem statement of the property with a `sorry` for the proof. Unlike existing FV benchmarks, each problem corresponds to real-world software written by engineers with no formal verification experience: as Table 1 shows, prior software-verification benchmarks are scarce (the bulk of ITP benchmarks target mathematics) and small, and none derive from code that was authored outside of a theorem-proving context, putting our problems out of distribution relative to anything an AI is likely to have memorized.

MvH: TODO: Summary of results.

The rest of the paper is organized as follows. We formalize the task in Section 2. We characterize our input corpus, detailing how we scraped and deduplicated $\approx 20k$ permissively-licensed Hypothesis tests from open-source repositories, in Section 3, and discuss how that corpus compares to a similar one released by the Hypothesis team [10]. Then, we describe our generation pipeline—scraping, dependency extraction, LLM-based transpilation with iterative Lean LSP repair, and post-production cleaning—in Section 4. We present baseline results in Section ??, discuss threats to validity in Section 5, survey related work in Section ??, and conclude in Section ??.

2 Task Definition

Each problem in the FVSpec benchmark consists of four components: (1) the original Python implementation of the function under test, (2) the original Python PBT exercising that implementation, (3) a Lean implementation of the same function—a fully defined, computable definition, no `sorry`—and (4) a Lean theorem corresponding to the PBT, with a `sorry` placeholder where the proof should go. The AI’s task is to fill in every theorem `sorry` with a Lean proof term that compiles cleanly under `lake build`; submissions are then scored by whether the proof retains any `sorry`, `admit`, or axioms beyond Mathlib’s defaults. The Lean kernel is the sole arbiter—we perform no separate semantic comparison against the original Python. Each problem also includes a URL pointing to the original source code and license information.

As a running example, consider `isort(1st)`. Figure 1 shows all four artifacts of a single FVSpec problem. The left column holds the original implementation (top) and the PBT exercising it (bottom); the right column holds the corresponding `Impl.lean` (top), a complete Lean port of the function, and `Spec.lean` (bottom), containing two theorems—one for sortedness, one for permutation—each with their own `sorry`. The AI’s task is to discharge both Lean `sorry` placeholders.

The `isort` example is deliberately pure; Section 5 covers how the pipeline handles effectful code (and where the approach has limits).

We encounter two practical issues when translating Python PBTs into Lean implementations and (unproven) specifications.

Nonsensical theorems. Our pipeline can emit well-formed but useless statements in three ways.

(i) *Trivial Unit-typed conjectures.* When the formalization agent cannot recover a function’s true return type, it falls back on `Unit` as a placeholder. The resulting theorem is a tautology—e.g. for our running example, `theorem isort_check : isort 1st = () := by plausible`, which holds for any total function into `Unit` and so carries no verification value.

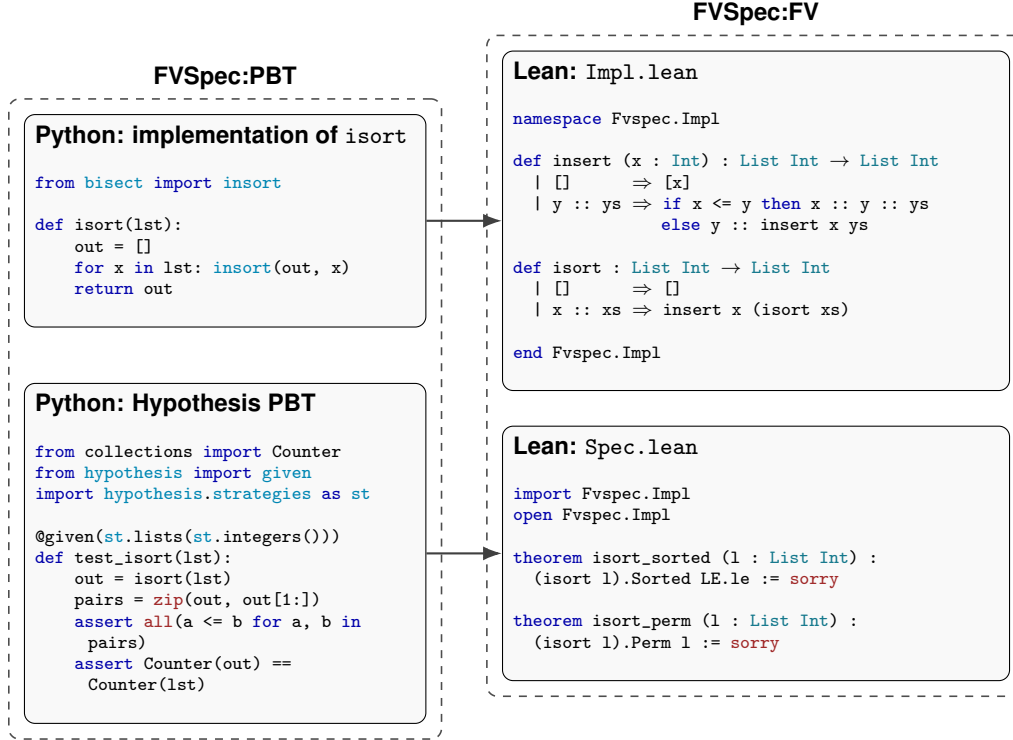


Figure 1: The four artifacts of a single FVSpec problem. Each problem is partitioned into the *PBT corpus* (FVSpec:PBT, left)—the original Python implementation and Hypothesis test we scrape from GitHub—and the *formal-verification challenge* (FVSpec:FV, right) we transpile from it: a complete Lean port of the implementation and a Lean theorem statement of the property under test. The AI must discharge both Lean sorry placeholders, one per theorem.

111 (ii) *Ill-typed or unresolved-symbol theorems.* The formalization agent works from the Python
 112 source and writes Lean by analogy. When it invents a Lean name that does not exist—for in-
 113 stance, writing theorem isort_perm (l : List Int) : (isort l).permutationOf l
 114 := sorry, where permutationOf is not a member of List (the correct name is List.Perm)—
 115 or attaches a type signature inconsistent with the surrounding code, the Lean elaborator rejects the
 116 file with errors like *unknown identifier*, *type mismatch*, *function expected*, or *failed to synthesize*.

117 (iii) *Spec theorems leaking into Impl.lean.* Our pipeline runs two separate agents: one writes
 118 Impl.lean (definitions only), the other writes Spec.lean (theorems only). The implementa-
 119 tion agent is explicitly instructed to emit no theorems, but occasionally—when shown the isort
 120 source—it pre-empts the spec agent by hallucinating a namespace Fvspec.Spec ... theorem
 121 isort_sorted ... end Fvspec.Spec block inside Impl.lean, which violates our file split and
 122 would silently expand the proof obligation if left in.

123 We catch (i) by regex, drop (ii) at compile time, and strip (iii) with a namespace-aware filter. A
 124 theorem that survives all three filters is a useful challenge problem even if it has drifted from the
 125 original PBT—because Lean checks proofs against *the theorem we generated*, not the informal
 126 property.

127 **Lean as a (nearly) perfect oracle.** The Lean kernel arbitrates correctness, which is qualitatively
 128 much stronger than statistical sampling. It is not, however, adversarially perfect: an AI can satisfy the
 129 kernel by *changing the question*—adding the Axiom of Choice, weakening hypotheses, or exploiting
 130 a security vulnerability in Lean itself [16]—and the proof will still type-check [32]. To make gaming
 131 visible, every submission is scored on two axes: a binary *proved* flag (the file compiles under lake
 132 build and every original sorry has been discharged) and partial credit (fraction of original sorries
 133 discharged). We additionally record any axiom dependencies the proof introduces, since an AI can
 134 trivialize a goal by quietly importing strong axioms; we do not currently auto-fail on extra axioms,

but report them so reviewers can flag suspicious solutions. For higher-stakes evaluations, submissions can additionally be re-checked through `leanprover/comparator` [24], which sandboxes the build and cross-validates the proof term against an independently-implemented Rust kernel. Lastly, a sufficiently motivated AI can still smuggle a vacuous proof past us when the original theorem statement is itself too weak—a limit shared by every benchmark whose scale exceeds full human review capacity.

Next, we explain how we scrape the PBT corpus and what the corpus looks like.

3 Property-Based Test Corpus (FVSpec:PBT)

We scrape real-world Python property-based tests from public GitHub repositories that depend on Hypothesis [31]. Our scraper operates in three stages. (1) *Discovery*. We crawl the Hypothesis dependency graph using PyGitHub [20] and persist 19,808 unique `owner/repo` pairs into a static snapshot. (2) *Licensing*. We skip repositories with an explicitly non-permissive license (e.g. AGPL, GPL, LGPL, MPL), and record the license string—including “unknown” for repositories with no detectable license—alongside the rest of each repository’s metadata. (3) *Extraction*. We shallow-clone each accepted repository, locate PBTs via `ripgrep`, and parse the surrounding Python to recover the transitive closure of locally-defined functions called therein.⁴

A 24-hour run produced 54,345 raw PBTs. The GitHub dependency graph surfaces many forks of the same upstream (e.g. hundreds of forks of `pytorch` carrying an identical Hypothesis suite), so we group repositories by base name, retain the most-starred “canonical” fork per group, and additionally drop tests whose whitespace-normalized SHA-256 already appears in the canonical set. After deduplication the working corpus is 11,039 PBTs across 333 repositories owned by 281 distinct GitHub users, spanning 303 distinct upstream project names.

3.1 Diversity

A perennial concern with scraped benchmarks is concentration: SWE-bench [22], for instance, draws all 2,294 of its task instances from just 12 repositories, with `django` alone supplying 37% and the top three (`django`, `sympy`, `scikit-learn`) supplying 64%. Our corpus is far more spread out—it draws from 333 repositories, no single one of which contributes more than 8.7% of the PBTs and whose top ten together contribute 58.5%—and most of those repositories are unstarred personal projects rather than curated showcase code (Table 2). The median PBT is just 13 LoC, but exercises around $3\times$ its own length in project code (see Figure 2).

To illustrate the diversity of our dataset, consider the result of drawing ten repositories from it uniformly at random:⁵ `sec-parser` (SEC-filings parser), `whenever` (datetime library), `LDAR_Sim` (methane leak-detection simulator), `bowtie` (JSON-schema test runner), `Ivy-Octernships-ML` (ML-framework scaffolding), `aaanalysis` (amino-acid analysis library), `shrinkray` (test-case shrinker), `logot` (log-output assertion library), `tokenizations` (NLP tokenisation), and `onnx-embedding` (ONNX-backed embedding library). This is broadly representative of real-world software engineering, as opposed to purely academic or competitive programming.

3.2 Comparison with the Hypothesis Corpus

While we were finalizing this work, the Hypothesis maintainers released their own scraped corpus [10] (HC-2026). The two corpora target different downstream uses—HC-2026 is built to study Hypothesis *runtime* behavior (entropy consumption, predicate firing, coverage), while ours is built to feed a Lean formalization pipeline—which drives the design differences summarized in Table 3. A consequence of HC-2026’s runtime focus is that it filters out as “invalid” any repository whose PBTs may not be directly runnable without some custom setup: code that depends on external services, API keys, environment variables, or live databases. Our pipeline imposes no such requirement, since we only need a PBT’s source code to translate it into Lean. Of our 333 repositories, 28 appear in HC-2026’s valid (executable) set and 111 (33%) appear in the pre-validity superset—meaning roughly 80 of

⁴We also scrape TypeScript PBTs written using `fast-check` [14]; however, we chose to scope our Lean extraction in this work to just the Python PBTs due to token budget. The TypeScript PBTs are included in our dataset for future use.

⁵Reproduce with `random.seed(13); random.sample([r for r in repos if r.stars >= 10], 10)` against the 333-repo deduplicated corpus; the star floor filters out personal forks.

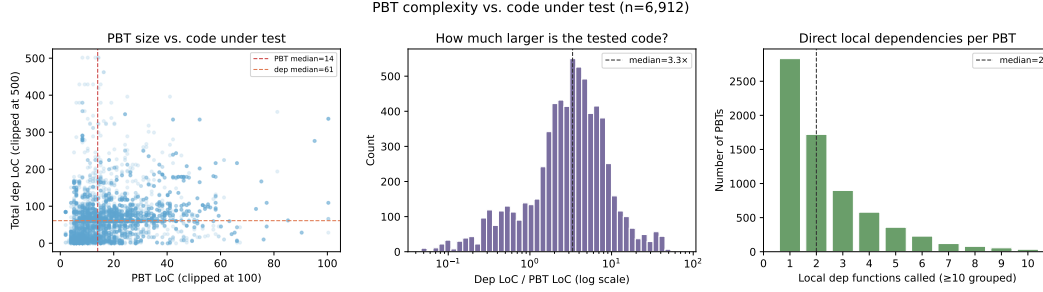


Figure 2: Three views of the relationship between PBT size and the size of the code being exercised, over the $n = 6,912$ post-deduplication PBTs whose local dependencies were recoverable. (a) Per-PBT scatter of PBT LoC against the total LoC of the directly-called dependencies. (b) Histogram of the ratio between dependency LoC and PBT LoC (log scale); the median PBT exercises code $3.3\times$ its own length. (c) Histogram of the number of direct local function calls per PBT.

Table 2: PBT corpus statistics.

Metric	Value
PBTs (raw scrape)	54,345
PBTs (post-dedup)	11,039
Unique repositories	333
Unique GitHub owners	281
Upstream project names	303
Top-1 / top-10 PBT share	8.7% / 58.5%
<i>Per-repo (min / med / max):</i>	
PBTs per repo	1 / 6 / 956
Stars	0 / 0 / 9,438
Forks	0 / 0 / 1,567
<i>Per-PBT (min / med / max):</i>	
PBT LoC	2 / 13 / 174
Direct fn deps	0 / 1 / 15

Table 3: Comparison with HC-2026 [10].

	FVSpec:PBT	HC-2026
Discovery	dep. graph	GitHub search
Raw PBTs	54,345	38,885
Tests (deduped)	11,039	23,139
Repos	333	1,504
Repo dedup	fork/name	is_fork
Test dedup	SHA-256	parametrization
License filter	no copyleft	none
Per-test data	source + metrics	runtime + coverage
Repo overlap (valid)	28 / 333	(8%)
Repo overlap (any)	111 / 333	(33%)

our repositories were dropped by HC-2026 for being too operationally entangled to run, consistent with our goal of producing challenging and realistic FV problems. Future work could merge the two corpora to enlarge the input pool; we did not pursue this here because the present token budget does not exhaust even our own 11,039-PBT corpus.

Next, we describe the pipeline by which we transpile our PBT corpus into Lean FV challenges.

4 FVSpec:PBT to FVSpec:FV Pipeline

We lift each PBT in the Section 3 corpus (FVSpec:PBT) into a Lean (`Impl.lean`, `Spec.lean`) pair (Figure 4, bottom) in three phases: function discovery, agentic transpilation, and post-production. We call the resulting FV challenge corpus FVSpec:FV. The pipeline is modeled on FVAPPS [13] but starts from PBTs rather than programming puzzles, and adds an iterative LSP-repair loop.

Function discovery. For each PBT we use tree-sitter to extract the function under test (FUT) and the transitive closure of its locally-defined dependencies. For our running example `isort`, this is just the four-line `def isort` body itself: `insort` comes from `bisect`, which is external and not in scope for transitive recovery. Larger PBTs in the corpus pull in tens to hundreds of lines of internal helpers (see Figure 2). When discovery fails—typically due to metaclasses, decorators, or other dynamic dispatch—we emit a stub FUT with generic type parameters so the formalization agent still receives well-formed input.

Agentic transpilation. A single formalization agent translates each sample, producing both `Impl.lean` (a computable port of the FUT and its dependencies, no sorry) and `Spec.lean` (one theorem per Hypothesis assertion, with proofs left as sorry). This is exemplified for `isort` in

Figure 1. After each generation step we type-check the output via the Lean LSP (over MCP) and `lake build`; on failure, the compiler diagnostics are fed back to the agent and a revised translation is requested. The loop terminates on success or after 16 tool-iterations.

Post-production. We first extract LLM turn counts from the raw `.eval` archives produced by the formalization agent, which runs as an Inspect AI [43] task. We then collapse runs into a single JSONL, deduplicating by sample id and tie-breaking with a quality score (structural faithfulness, theorem count, unit-tests, implementation success). Next, we re-run `lake build` on each retained sample and drop failures, which we classify into nine families: timeouts, well-founded recursion, unknown identifiers, failures to synthesize, type mismatches, syntax errors, missing functions, application errors, and declaration errors. We then run `radon` on the Python source and a custom AST analyzer on the Lean artifacts to extract line counts, theorem and def counts, residual `sorry/admits`, and axioms. Finally, conditioning on these metrics, we assign a binary easy/hard label using `claude-haiku-4-5` as judge, together with a calibrated confidence score and a short rationale.

The structural-faithfulness score used in the merge tie-break is a fixed weighted average of five sub-scores, each measuring how much of a Python-side artifact survives translation: parameter coverage (0.25), type correspondence (0.25), strategy coverage (0.20), assertion coverage (0.20), and dependency coverage (0.10). For `isort`, parameter coverage rewards the Lean theorem taking a single `List Int` matching Python’s `lst`; type correspondence verifies that Python’s `list[int]` maps to `List Int`; strategy coverage tracks that `st.lists(st.integers())` ranges over arbitrary integer lists, with no missing bounds in the Lean version; assertion coverage rewards both Python assertions producing a corresponding Lean theorem; and dependency coverage rewards every named helper recovered in function discovery surviving into `Impl.lean`. Sub-scores ship alongside the overall score so consumers can re-weight or filter.

4.1 Pipeline yield

The published FVSpec:FV dataset contains **9,415 samples** drawn from **7,850 distinct PBTs across 250 repositories** (Figure 4, bottom right): formalization succeeded for $7,850/11,039 = 71\%$ of the input PBTs and $250/333 = 75\%$ of the input repositories, with the rest dropped by `lake build`. The mean sample carries 8 theorems (median 6, max 66) and median structural faithfulness 0.65; the full distribution and per-sub-metric means are in Figure 3. Implementation autoformalization succeeded for 83% of samples; the remaining 17% ship against a stub `Impl.lean`, so the theorem statement is well-typed but the body it constrains is opaque. The Claude Haiku grader splits the dataset 62%/38% hard/easy. Where a PBT yields multiple compiling formalizations (1,348 PBTs, mean 2.2 attempts each), all are kept; the `is_canonical` field marks the highest-quality version per PBT.

5 Threats to Validity

The most consequential threat to FVSpec’s validity is whether theorems stated over uninterpreted axioms actually mean anything about the original Python. We address this head-on in Section 5.1, then catalog the remaining threats.

5.1 Axiomatized interfaces for effectful code

The `isort` running example from Section 2 is deliberately pure. Most PBTs in our corpus are not: production code routinely touches Redis, sockets, the filesystem, the wall clock, webhook queues, and so forth. A natural objection is that any Lean theorem extracted from such a PBT cannot say anything meaningful about the live system.

The formalization agent handles this by treating the effectful world as an *uninterpreted interface*. Parts of the source essential to the property—the entities and operators it constrains—are translated into Lean structures and inductive types. Parts that are pure plumbing—the database, the queue, the system clock—are introduced as opaque `axiom` declarations and threaded through the theorem as parameters. The resulting theorem holds *over all interpretations* of those axioms, which is exactly the contract the original PBT was checking against the live environment.

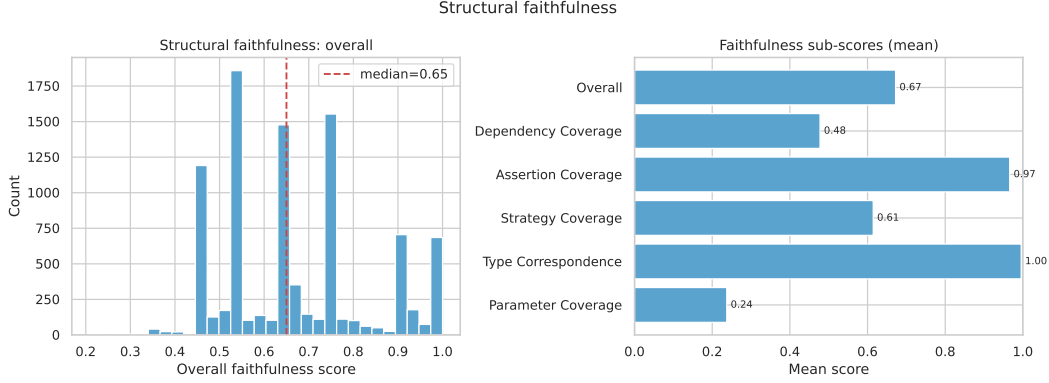


Figure 3: Structural-faithfulness scores across the published FVSpec:FV dataset. *Left*: histogram of per-sample overall scores (median 0.65; the multi-modal shape reflects the discrete denominators of the underlying ratios). *Right*: mean per-sub-metric scores. Type correspondence (1.00) and assertion coverage (0.97) are near-saturated—most translations preserve types and predicates—while parameter coverage (0.24) and dependency coverage (0.48) are the weak links, dragging the overall mean down to 0.67. The high variance in per-sample scores is exactly what the `is_canonical` flag exploits when multiple formalizations of the same PBT are available.

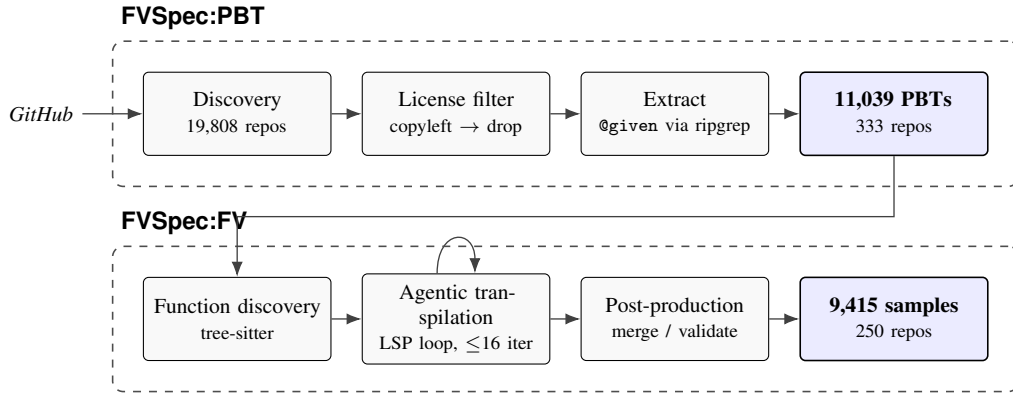


Figure 4: The full FVSpec generation pipeline. *Top* (FVSpec:PBT): the input PBT corpus is scraped from public GitHub repositories (Section 3). *Bottom* (FVSpec:FV): each PBT is lifted into a Lean (Impl, Spec) pair via tree-sitter-based function discovery, an agentic transpilation step (with the LSP-repair self-loop), and a deterministic post-production stage that validates, deduplicates, and grades the output.

251 A canonical instance is `test_workflow_job_time_to_start` from `scality/runner-manager`⁶,
 252 a GitHub Actions self-hosted runner controller. The Python PBT initializes Redis-backed model
 253 classes, runs an ORM migrator, calls `datetime.now()` to fabricate two timestamps straddling a
 254 runner-startup timeout, enqueues a webhook through a background job queue, and asserts on the
 255 resulting runner count. The FVSpec translation introduces seventeen axioms—including Redis,
 256 Queue, State, State.after_enqueue, and get_runners—and states the contract purely in terms
 257 of these uninterpreted operators:

```

258 theorem create_runner_when_above_timeout
259   (initial_state : State) (state_after_enqueue : State)
260   (runner_group : RunnerGroup) (webhook_updated : WorkflowJobEvent)
261   (settings : ExtendedSettings) (created_at started_at : Time)
262   (h_initial_count : (get_runners runner_group initial_state).length = 1)
263   (h_below_max : 1 < runner_group.max)
264   (h_above_timeout : started_at - created_at > settings.timeout_runner)
265   (h_enqueue : state_after_enqueue =
266     State.after_enqueue initial_state runner_group webhook_updated settings)

```

⁶<https://github.com/scality/runner-manager>

267 : (get_runners runner_group state_after_enqueue).length = 2 := sorry

268 A model proving `create_runner_when_above_timeout` is reasoning about a scheduler’s state-
269 transition semantics under a timeout precondition; the fact that the production scheduler talks to
270 Redis is no more load-bearing in the theorem than the choice of register allocator is in a correctness
271 proof of a sorting algorithm.

272 5.2 Where axiomatization breaks down

273 The axiomatization move is not a universal solvent.

274 *Properties about the effect itself.* If the property is fundamentally about timing, memory, or-
275 dering of side effects, or wall-clock latency, an uninterpreted axiom cannot witness it. `assert`
276 `response_time < 100ms` cannot become a meaningful Lean theorem because the relevant seman-
277 tics is precisely what the axiomatization discards.

278 *Effectful code disguised as pure code.* The agent identifies effectful boundaries via type signatures
279 and call patterns. When a function has a pure signature but performs side effects internally (e.g.,
280 a logger called via a global, an `lru_cache`-style hidden state), the agent will inline it as if pure.
281 The resulting theorem may compile and even be true, but its proof obligation does not constrain the
282 original side-effecting behavior.

283 *Axioms that misrepresent the API contract.* `axiom Redis` : Type captures *some* Redis but not the
284 real one. The agent does not encode Redis’s consistency guarantees, ordering, or failure modes. A
285 theorem proven over the axiomatized interface may not transfer to the live system if the production
286 code relies on those omitted guarantees.

287 5.3 Other threats

288 *Translation faithfulness.* Specifications are translated rather than hand-written, so some may not
289 faithfully represent the original PBT. Our structural-faithfulness score (Section 4) is a heuristic: a
290 high score does not guarantee semantic equivalence, and a low score does not necessarily indicate a
291 useless problem.

292 *LLM grader.* Difficulty labels come from `claude-haiku-4-5`, itself a model whose judgments are
293 imperfect and uncalibrated against human expert performance. We expose the grader’s confidence
294 and rationale per sample so consumers can audit or override the labels.

295 *Stub-implementation tail.* Implementation autoformalization succeeds for 83% of samples (Section 4).
296 The remaining 17% ship against a stub `Impl`. Lean, weakening the proof obligation: the theorem is
297 well-typed, but the body it constrains is opaque. We tag these samples so consumers can filter them
298 out.

299 *License-detection edge cases.* Our scraper (Section 3) skips repositories with explicitly non-permissive
300 licenses but accepts repositories where GitHub cannot detect a license at all. Most such cases are
301 forks of permissively-licensed upstreams (e.g., `pytorch`, `ivy`) where the `LICENSE` file was modified
302 or moved during forking, but we cannot guarantee this for every retained sample.

303 *No human baselines.* We do not measure human expert performance on FVSpec problems and
304 therefore cannot calibrate the dataset’s difficulty distribution against the proof effort required of
305 skilled humans.

306 References

- 307 [1] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad. Proofnet:
308 Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint*
309 *arXiv:2302.12433*, 2023.
- 310 [2] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. Holist: An environment for machine
311 learning of higher order logic theorem proving. In *International conference on machine learning*,
312 pages 454–463. PMLR, 2019.

- [3] S. Barkallah, S. Daruru, B. Miranda, L. Aniva, A. Nie, and S. Koyejo. VeriBench-FTP: A formal theorem proving benchmark in Lean 4 for code verification. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025.
- [4] S. Buro, R. Crole, and I. Mastroeni. Equational logic and categorical semantics for multi-languages. *Electronic Notes in Theoretical Computer Science*, 352:79–103, 2020.
- [5] S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, et al. A benchmark for vericoding: formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025.
- [6] S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [7] D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- [8] N. A. David Dalrymple. Safeguarded ai. <https://aria.org.uk/opportunity-spaces/mathematics-for-safe-ai/safeguarded-ai/>, 2025. Accessed 25 April 2026.
- [9] A. de Almeida Borges, A. Casanueva Artís, J.-R. Falleri, E. J. Gallego Arias, É. Martin-Dorel, K. Palmkog, A. Serebrenik, and T. Zimmermann. Lessons for interactive theorem proving researchers from a survey of coq users. *Journal of Automated Reasoning*, 69(1):8, 2025.
- [10] L. DeVoe. The Hypothesis corpus. <https://hypothesis.works/articles/hypothesis-corpus/>, 2026. Accessed April 2026.
- [11] P. C. Dillinger, P. Manolios, D. Vroon, and J. S. Moore. Acl2s:“the acl2 sedan”. *Electronic Notes in Theoretical Computer Science*, 174(2):3–18, 2007.
- [12] M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022. doi: 10.1109/MSEC.2022.3153035.
- [13] Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.
- [14] N. Dubien. fast-check: Property-based testing framework for JavaScript/TypeScript. <https://github.com/dubzzz/fast-check>, 2024. Accessed April 2026.
- [15] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järvinen, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla, and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.
- [16] K. Gopinathan. Lean proved this program was correct; then I found a bug. <https://kirancodes.me/posts/log-who-watches-the-watchers.html>, Apr. 2026. Blog post; reports a heap buffer overflow in `lean_alloc_sarray` in the Lean 4 runtime, found by AFL++ fuzzing of the verified `lean-zip` library. Accessed 26 April 2026.
- [17] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [18] J. Hu, T. Zhu, and S. Welleck. minictx: Neural theorem proving with (long-) contexts. *arXiv preprint arXiv:2408.03350*, 2024.

- [19] Y. Huang, X. Lin, Z. Liu, Q. Cao, H. Xin, H. Wang, Z. Li, L. Song, and X. Liang. Mustard: Mastering uniform synthesis of theorem and proof data. *arXiv preprint arXiv:2402.08957*, 2024.
- [20] V. Jacques, L. Wan, S. Kowalik, and E. Minack. PyGitHub. <https://github.com/PyGithub/PyGithub>, 2026. Accessed 26 April 2026.
- [21] J. Jiang, W. He, Y. Wang, G. Gao, Y. Hu, J. Wang, N. Guan, P. Wu, C. Dai, L. Xiao, and B. Dong. FATE: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- [22] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [23] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 207–220, 2009.
- [24] Lean FRO. Comparator: A trustworthy judge for Lean proofs. <https://github.com/leanprover/comparator>, 2026. Sandboxes the build and replays proof terms, optionally cross-checking against an independently-implemented Rust kernel (nanoda). Accessed 26 April 2026.
- [25] X. Leroy, S. Blazy, D. Kästner, B. Schommer, M. Pister, and C. Ferdinand. CompCert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [26] S. Li, W. Lu, X. Shi, K. Weng, H. Sun, M. Yu, T. Zhang, G. Yu, H. Liu, and L. Du. MSC-180: A benchmark for automated formal theorem proving from mathematical subject classification. *arXiv preprint arXiv:2512.18256*, 2025.
- [27] C. Liu, J. Shen, H. Xin, Z. Liu, Y. Yuan, H. Wang, W. Ju, C. Zheng, Y. Yin, L. Li, et al. Fimo: A challenge formal dataset for automated theorem proving. *arXiv preprint arXiv:2309.04295*, 2023.
- [28] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.
- [29] J. Lu, Y. Wan, Z. Liu, Y. Huang, J. Xiong, C. Liu, J. Shen, H. Jin, J. Zhang, H. Wang, et al. Process-driven autoformalization in lean 4. *arXiv preprint arXiv:2406.01940*, 2024.
- [30] P. Lu, J. Sheng, L. Lyu, J. Jin, T. Xia, A. Gu, and J. Zou. Solving inequality proofs with large language models. In *The 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [31] D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.
- [32] Q. D. Max von Hippel. Lies, damned lies, and proofs: Formal methods are not slopless. <https://www.lesswrong.com/posts/rhAPh3YzhPoBNpgHg/lies-damned-lies-and-proofs-formal-methods-are-not-slopless>, Jan. 2026. Accessed 26 April 2026.
- [33] E. Miyazono. Formally scalable AI oversight through specifications. <https://foresight.org/resource/evan-miyazono-formally-scalable-ai-oversight-through-specifications/>, 2024. Accessed 25 April 2026.
- [34] J. T. Perconti and A. Ahmed. Verifying an open compiler using multi-language semantics. In *European Symposium on Programming Languages and Systems*, pages 128–148. Springer, 2014.

- [35] A. Poiroux, G. Weiss, V. Kunčák, and A. Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17958–17980, 2025.
- [36] J. Rasmussen. Risk management in a dynamic society: a modelling problem. *Safety science*, 27(2-3):183–213, 1997.
- [37] Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [38] C. Song, Z. Wang, F. Pu, H. Wang, X. Lin, J. Liu, J. Li, and Z. Liu. LeanGeo: Formalizing computational geometry problems in Lean. *arXiv preprint arXiv:2508.14644v1*, 2025.
- [39] R. S. Team. 2025 state of rust survey results. <https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/>, March 2026. Accessed 25 April 2026.
- [40] A. Thakur, J. Lee, G. Tsoukalas, M. Sistla, M. Zhao, S. Zetsche, G. Durrett, Y. Yue, and S. Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.
- [41] K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner, and E. First. Rango: Adaptive retrieval-augmented proving for automated software verification. *arXiv preprint arXiv:2412.14063*, 2024.
- [42] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- [43] UK AI Security Institute. Inspect AI: Framework for large language model evaluations, 2024. URL https://github.com/UKGovernmentBEIS/inspect_ai.
- [44] M. von Hippel. <https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program>, 2026. Accessed 25 April 2026.
- [45] Z. Wu, J. Wang, D. Lin, and K. Chen. Lean-github: Compiling github lean repositories for a versatile Lean prover. *arXiv preprint arXiv:2407.17227*, 2024.
- [46] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. *Advances in Neural Information Processing Systems*, 37:105848–105863, 2024.
- [47] Z. Yu, R. Peng, K. Ding, Y. Li, Z. Peng, M. Liu, Y. Zhang, Z. Yuan, H. Xin, W. Huang, et al. Formalmath: Benchmarking formal mathematical reasoning of large language models. *arXiv preprint arXiv:2505.02735*, 2025.
- [48] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.